# TurnoFlex

## Documentación de API

Proyecto de Desarrollo Backend

2do Cuatrimestre 2026


**Integrantes**

Mateo Israel Adan
Baca Fernando Ezequiel
Lamas Carmen Leonela
Mata Botana Raúl Alberto
Yrala Marcelo

# 1. Presentación del proyecto

TurnoFlex es un backend destinado a la gestión de turnos de un centro de atención. Permite administrar profesionales, clientes, disponibilidades horarias y turnos, incorporando validaciones y reglas de negocio para evitar reservas duplicadas y conflictos de horarios.

El proyecto reemplaza el manejo basado en planillas y mensajes. De esta manera se busca prevenir turnos duplicados, controlar los horarios disponibles y facilitar la consulta y cancelación de reservas.

## 2. Tecnologías utilizadas

- Node.js.
- Express.
- JavaScript.
- Pug.
- JSON como persistencia.
- async/await y Promises.
- Módulos CommonJS.
- POO mediante clases.
- Thunder Client para pruebas.

## 3. Arquitectura

El proyecto está organizado por responsabilidades:

- routes/: define los endpoints y los relaciona con los controladores.
- controllers/: recibe las solicitudes, realiza validaciones, aplica reglas de negocio y construye las respuestas.
- models/: contiene las clases de las entidades principales.
- middlewares/: contiene el logging de solicitudes y el manejo general de errores.
- utils/jsonFile.js: centraliza la lectura y escritura asíncrona del archivo JSON.
- data/db.json: almacena la información persistente de la aplicación.
- views/: contiene la plantilla Pug utilizada por la vista informativa.

## 4. Recursos del sistema

### Profesional

Campos:

- id
- nombre
- apellido

### Cliente

Campos:

- id
- nombre
- apellido

### Disponibilidad

Campos:

- id
- profesionalId
- fecha
- hora
- disponible

### Turno

Campos:

- id
- profesionalId
- clienteId
- disponibilidadId
- fecha
- hora
- estado

Estados permitidos:

- reservado
- cancelado
- atendido

## 5. Reglas de negocio

- Un profesional no puede tener dos turnos reservados en el mismo horario.
- Un cliente no puede reservar dos veces el mismo horario.
- Solo pueden reservarse disponibilidades cuyo valor disponible sea true.
- Un turno se crea inicialmente con estado reservado.
- La transición reservado → atendido se realiza mediante PUT.
- La transición reservado → cancelado se realiza mediante PATCH /turnos/:id/cancelar.

- Cancelar libera nuevamente la disponibilidad.
- Atender mantiene la disponibilidad ocupada.
- No se puede cancelar dos veces el mismo turno.
- No se puede atender un turno cancelado.

## 6. Endpoints

Las respuestas correctas se muestran con sus códigos habituales. En los ejemplos se utiliza localhost:3000 como servidor.

### Ruta principal

*GET /*

- **Recurso:** API TurnoFlex.
- **Método HTTP:** GET.
- **URL:** /.
- **Parámetros de ruta:** no tiene.
- **Parámetros de consulta:** no tiene.
- **Body:** no corresponde.
- **Descripción:** comprueba que la API está funcionando.
- **Ejemplo de solicitud:**

```
GET http://localhost:3000/
```

- **Código HTTP esperado:** 200 OK.
- **Respuesta correcta:**

```
{
  "mensaje": "La API TurnoFlex funciona correctamente"
}
```

### Vista informativa

*GET /vista*

- **Recurso:** vista informativa.
- **Método HTTP:** GET.
- **URL:** /vista.
- **Parámetros de ruta:** no tiene.
- **Parámetros de consulta:** no tiene.
- **Body:** no corresponde.
- **Descripción:** renderiza la plantilla Pug informativa de TurnoFlex.
- **Ejemplo de solicitud:**

```
GET http://localhost:3000/vista
```

- **Código HTTP esperado:** 200 OK.
- **Respuesta correcta:** documento HTML generado desde src/views/index.pug.
- **Ejemplo de error:** ante un error inesperado de renderización, el middleware general responde:

```
{
  "mensaje": "Error interno del servidor"
}
```

- **Código HTTP del error:** 500 Internal Server Error.

## Profesionales

*GET /profesionales*

- **Recurso:** profesionales.
- **Método HTTP:** GET.
- **URL:** /profesionales.
- **Parámetros de ruta:** no tiene.
- **Parámetros de consulta:** no tiene.
- **Body:** no corresponde.
- **Descripción:** lista todos los profesionales.
- **Ejemplo de solicitud:**

```
GET http://localhost:3000/profesionales
```

- **Código HTTP esperado:** 200 OK.
- **Respuesta correcta:**

```
[
  {
    "id": 1,
    "nombre": "Laura",
    "apellido": "Gomez"
  }
]
```

- **Ejemplo de error:** error inesperado de lectura del archivo:

```
{
  "mensaje": "Error interno del servidor"
}
```

- **Código HTTP del error:** 500 Internal Server Error.

*GET /profesionales/:id*

- **Recurso:** profesional individual.
- **Método HTTP:** GET.
- **URL:** /profesionales/:id.
- **Parámetros de ruta:** id, identificador del profesional.

- **Parámetros de consulta:** no tiene.
- **Body:** no corresponde.
- **Descripción:** obtiene un profesional por su ID.
- **Ejemplo de solicitud:**

```
GET http://localhost:3000/profesionales/1
```

- **Código HTTP esperado:** 200 OK.
- **Respuesta correcta:**

```
{
  "id": 1,
  "nombre": "Laura",
  "apellido": "Gomez"
}
```

- **Ejemplo de error:**

```
{
  "mensaje": "Profesional no encontrado"
}
```

- **Código HTTP del error:** 404 Not Found.

*POST /profesionales*

- **Recurso:** profesional.
- **Método HTTP:** POST.
- **URL:** /profesionales.
- **Parámetros de ruta:** no tiene.
- **Parámetros de consulta:** no tiene.
- **Body:** JSON con nombre y apellido.
- **Descripción:** crea un profesional y genera automáticamente su ID.
- **Ejemplo de solicitud:**

```
POST http://localhost:3000/profesionales
Content-Type: application/json

{
  "nombre": "Laura",
  "apellido": "Gomez"
}
```

- **Código HTTP esperado:** 201 Created.
- **Respuesta correcta:**

```
{
  "id": 1,
  "nombre": "Laura",
  "apellido": "Gomez"
}
```

- **Ejemplo de error:**

```
{
  "mensaje": "nombre y apellido son obligatorios y deben ser strings no vacios"
}
```

- **Código HTTP del error:** 400 Bad Request.

*PUT /profesionales/:id*

- **Recurso:** profesional.
- **Método HTTP:** PUT.
- **URL:** /profesionales/:id.
- **Parámetros de ruta:** id, identificador del profesional.
- **Parámetros de consulta:** no tiene.
- **Body:** JSON con nombre y apellido.
- **Descripción:** modifica los datos de un profesional existente.
- **Ejemplo de solicitud:**

```
PUT http://localhost:3000/profesionales/1
Content-Type: application/json

{
  "nombre": "Laura",
  "apellido": "Gomez"
}
```

- **Código HTTP esperado:** 200 OK.
- **Respuesta correcta:**

```
{
  "id": 1,
  "nombre": "Laura",
  "apellido": "Gomez"
}
```

- **Ejemplo de error:**

```
{
  "mensaje": "Profesional no encontrado"
}
```

- **Código HTTP del error:** 404 Not Found.
- **Datos inválidos:** responden 400 Bad Request.

*DELETE /profesionales/:id*

- **Recurso:** profesional.
- **Método HTTP:** DELETE.
- **URL:** /profesionales/:id.
- **Parámetros de ruta:** id, identificador del profesional.

- **Parámetros de consulta:** no tiene.
- **Body:** no corresponde.
- **Descripción:** elimina un profesional.
- **Ejemplo de solicitud:**

```
DELETE http://localhost:3000/profesionales/1
```

- **Código HTTP esperado:** 204 No Content.
- **Respuesta correcta:** sin contenido.
- **Ejemplo de error:**

```
{
  "mensaje": "Profesional no encontrado"
}
```

- **Código HTTP del error:** 404 Not Found.

## Clientes

*GET /clientes*

- **Recurso:** clientes.
- **Método HTTP:** GET.
- **URL:** /clientes.
- **Parámetros de ruta:** no tiene.
- **Parámetros de consulta:** no tiene.
- **Body:** no corresponde.
- **Descripción:** lista todos los clientes.
- **Ejemplo de solicitud:**

```
GET http://localhost:3000/clientes
```

- **Código HTTP esperado:** 200 OK.
- **Respuesta correcta:**

```
[
  {
    "id": 1,
    "nombre": "Lucia",
    "apellido": "Rodriguez"
  }
]
```

- **Ejemplo de error:** error inesperado de lectura:

```
{
  "mensaje": "Error interno del servidor"
}
```

- **Código HTTP del error:** 500 Internal Server Error.

*GET /clientes/:id*

- **Recurso:** cliente individual.
- **Método HTTP:** GET.
- **URL:** /clientes/:id.
- **Parámetros de ruta:** id, identificador del cliente.
- **Parámetros de consulta:** no tiene.
- **Body:** no corresponde.
- **Descripción:** obtiene un cliente por su ID.
- **Ejemplo de solicitud:**

```
GET http://localhost:3000/clientes/1
```

- **Código HTTP esperado:** 200 OK.
- **Respuesta correcta:**

```
{
  "id": 1,
  "nombre": "Lucia",
  "apellido": "Rodriguez"
}
```

- **Ejemplo de error:**

```
{
  "mensaje": "Cliente no encontrado"
}
```

- **Código HTTP del error:** 404 Not Found.

*POST /clientes*

- **Recurso:** cliente.
- **Método HTTP:** POST.
- **URL:** /clientes.
- **Parámetros de ruta:** no tiene.
- **Parámetros de consulta:** no tiene.
- **Body:** JSON con nombre y apellido.
- **Descripción:** crea un cliente y genera automáticamente su ID.
- **Ejemplo de solicitud:**

```
POST http://localhost:3000/clientes
Content-Type: application/json

{
  "nombre": "Lucia",
  "apellido": "Rodriguez"
}
```

- **Código HTTP esperado:** 201 Created.

- **Respuesta correcta:**

```
{
 "id": 1,
 "nombre": "Lucia",
 "apellido": "Rodriguez"
}
```

- **Ejemplo de error:**

```
{
 "mensaje": "nombre y apellido son obligatorios y deben ser strings no vacios"
}
```

- **Código HTTP del error:** 400 Bad Request.

*PUT /clientes/:id*

- **Recurso:** cliente.
- **Método HTTP:** PUT.
- **URL:** /clientes/:id.
- **Parámetros de ruta:** id, identificador del cliente.
- **Parámetros de consulta:** no tiene.
- **Body:** JSON con nombre y apellido.
- **Descripción:** modifica los datos de un cliente existente.
- **Ejemplo de solicitud:**

```
PUT http://localhost:3000/clientes/1
Content-Type: application/json

{
 "nombre": "Lucia",
 "apellido": "Rodriguez"
}
```

- **Código HTTP esperado:** 200 OK.
- **Respuesta correcta:**

```
{
 "id": 1,
 "nombre": "Lucia",
 "apellido": "Rodriguez"
}
```

- **Ejemplo de error:**

```
{
 "mensaje": "Cliente no encontrado"
}
```

- **Código HTTP del error:** 404 Not Found.
- **Datos inválidos:** responden 400 Bad Request.

*DELETE /clientes/:id*

- **Recurso:** cliente.
- **Método HTTP:** DELETE.
- **URL:** /clientes/:id.
- **Parámetros de ruta:** id, identificador del cliente.
- **Parámetros de consulta:** no tiene.
- **Body:** no corresponde.
- **Descripción:** elimina un cliente.
- **Ejemplo de solicitud:**

```
DELETE http://localhost:3000/clientes/1
```

- **Código HTTP esperado:** 204 No Content.
- **Respuesta correcta:** sin contenido.
- **Ejemplo de error:**

```
{
  "mensaje": "Cliente no encontrado"
}
```

- **Código HTTP del error:** 404 Not Found.

## Disponibilidades

*GET /disponibilidades*

- **Recurso:** disponibilidades.
- **Método HTTP:** GET.
- **URL:** /disponibilidades.
- **Parámetros de ruta:** no tiene.
- **Parámetros de consulta:** opcionales:
  - profesionalId
  - fecha
  - disponible
- **Body:** no corresponde.
- **Descripción:** lista disponibilidades y permite aplicar filtros combinados.
- **Ejemplo de solicitud:**

```
GET http://localhost:3000/disponibilidades?profesionalId=1&fecha=2026-10-15&disponible=true
```

- **Código HTTP esperado:** 200 OK.
- **Respuesta correcta:**

```
[
  {
```

```
  "id": 1,
  "profesionalId": 1,
  "fecha": "2026-10-15",
  "hora": "11:00",
  "disponible": true
 }
]
```


- **Ejemplo de error:**


```
{
 "mensaje": "Filtros incorrectos"
}
```


- **Código HTTP del error:** 400 Bad Request.

*GET /disponibilidades/:id*

- **Recurso:** disponibilidad individual.
- **Método HTTP:** GET.
- **URL:** /disponibilidades/:id.
- **Parámetros de ruta:** id, identificador de la disponibilidad.
- **Parámetros de consulta:** no tiene.
- **Body:** no corresponde.
- **Descripción:** obtiene una disponibilidad por su ID.
- **Ejemplo de solicitud:**

```
GET http://localhost:3000/disponibilidades/1
```

- **Código HTTP esperado:** 200 OK.
- **Respuesta correcta:**


```
{
 "id": 1,
 "profesionalId": 1,
 "fecha": "2026-10-15",
 "hora": "11:00",
 "disponible": true
}
```


- **Ejemplo de error:**


```
{
 "mensaje": "Disponibilidad no encontrada"
}
```


- **Código HTTP del error:** 404 Not Found.

*POST /disponibilidades*

- **Recurso:** disponibilidad.
- **Método HTTP:** POST.
- **URL:** /disponibilidades.

- **Parámetros de ruta:** no tiene.
- **Parámetros de consulta:** no tiene.
- **Body:** JSON con profesionalId, fecha y hora. disponible es opcional.
- **Descripción:** crea una disponibilidad asociada a un profesional. Si no se envía disponible, se establece en true.
- **Ejemplo de solicitud:**

```
POST http://localhost:3000/disponibilidades
Content-Type: application/json

{
  "profesionalId": 1,
  "fecha": "2026-10-15",
  "hora": "11:00"
}
```

- **Código HTTP esperado:** 201 Created.
- **Respuesta correcta:**

```
{
  "id": 1,
  "profesionalId": 1,
  "fecha": "2026-10-15",
  "hora": "11:00",
  "disponible": true
}
```

- **Ejemplo de error — profesional inexistente:**

```
{
  "mensaje": "Profesional no encontrado"
}
```

- **Código HTTP del error:** 404 Not Found.
- **Disponibilidad duplicada:** responde 409 Conflict.
- **Datos inválidos:** responde 400 Bad Request.

*PUT /disponibilidades/:id*

- **Recurso:** disponibilidad.
- **Método HTTP:** PUT.
- **URL:** /disponibilidades/:id.
- **Parámetros de ruta:** id, identificador de la disponibilidad.
- **Parámetros de consulta:** no tiene.
- **Body:** JSON con profesionalId, fecha, hora y disponible.
- **Descripción:** modifica una disponibilidad existente.
- **Ejemplo de solicitud:**

PUT http://localhost:3000/disponibilidades/1
Content-Type: application/json

```
{
 "profesionalId": 1,
 "fecha": "2026-10-15",
 "hora": "11:00",
 "disponible": false
}
```

- **Código HTTP esperado:** 200 OK.
- **Respuesta correcta:**

```
{
 "id": 1,
 "profesionalId": 1,
 "fecha": "2026-10-15",
 "hora": "11:00",
 "disponible": false
}
```

- **Ejemplo de error:**

```
{
 "mensaje": "Disponibilidad no encontrada"
}
```

- **Código HTTP del error:** 404 Not Found.
- **Datos inválidos:** 400 Bad Request.
- **Disponibilidad duplicada:** 409 Conflict.

*DELETE /disponibilidades/:id*

- **Recurso:** disponibilidad.
- **Método HTTP:** DELETE.
- **URL:** /disponibilidades/:id.
- **Parámetros de ruta:** id, identificador de la disponibilidad.
- **Parámetros de consulta:** no tiene.
- **Body:** no corresponde.
- **Descripción:** elimina una disponibilidad.
- **Ejemplo de solicitud:**

DELETE http://localhost:3000/disponibilidades/1

- **Código HTTP esperado:** 204 No Content.
- **Respuesta correcta:** sin contenido.
- **Ejemplo de error:**

```
{
 "mensaje": "Disponibilidad no encontrada"
}
```

- **Código HTTP del error:** 404 Not Found.

## Turnos

*GET /turnos*

- **Recurso:** turnos.
- **Método HTTP:** GET.
- **URL:** /turnos.
- **Parámetros de ruta:** no tiene.
- **Parámetros de consulta:** opcionales:
  - profesionalId
  - clienteId
  - fecha
  - estado
- **Body:** no corresponde.
- **Descripción:** lista turnos y permite filtrar por profesional, cliente, fecha y estado.
- **Ejemplo de solicitud:**

```
GET http://localhost:3000/turnos?profesionalId=1&estado=reservado
```

- **Código HTTP esperado:** 200 OK.
- **Respuesta correcta:**

```
[
 {
  "id": 1,
  "profesionalId": 1,
  "clienteId": 1,
  "disponibilidadId": 1,
  "fecha": "2026-10-15",
  "hora": "11:00",
  "estado": "reservado"
 }
]
```

- **Ejemplo de error:**

```
{
 "mensaje": "Filtros incorrectos"
}
```

- **Código HTTP del error:** 400 Bad Request.

*GET /turnos/:id*

- **Recurso:** turno individual.
- **Método HTTP:** GET.
- **URL:** /turnos/:id.

- **Parámetros de ruta:** id, identificador del turno.
- **Parámetros de consulta:** no tiene.
- **Body:** no corresponde.
- **Descripción:** obtiene un turno por su ID.
- **Ejemplo de solicitud:**

```
GET http://localhost:3000/turnos/1
```

- **Código HTTP esperado:** 200 OK.
- **Respuesta correcta:**

```
{
 "id": 1,
 "profesionalId": 1,
 "clienteId": 1,
 "disponibilidadId": 1,
 "fecha": "2026-10-15",
 "hora": "11:00",
 "estado": "reservado"
}
```

- **Ejemplo de error:**

```
{
 "mensaje": "Turno no encontrado"
}
```

- **Código HTTP del error:** 404 Not Found.

*POST /turnos*

- **Recurso:** turno.
- **Método HTTP:** POST.
- **URL:** /turnos.
- **Parámetros de ruta:** no tiene.
- **Parámetros de consulta:** no tiene.
- **Body:** únicamente profesionalId, clienteId y disponibilidadId.
- **Descripción:** reserva una disponibilidad y crea un turno. fecha y hora se obtienen de la disponibilidad. El estado se genera automáticamente como reservado.
- **Ejemplo de solicitud:**

```
POST http://localhost:3000/turnos
Content-Type: application/json

{
 "profesionalId": 1,
 "clienteId": 1,
 "disponibilidadId": 1
}
```

- **Código HTTP esperado:** 201 Created.

- **Respuesta correcta:**


```
{
 "id": 1,
 "profesionalId": 1,
 "clienteId": 1,
 "disponibilidadId": 1,
 "fecha": "2026-10-15",
 "hora": "11:00",
 "estado": "reservado"
}
```


- **Ejemplos de errores:**

| Situación | Código |
|---|---|
| Datos inválidos | 400 Bad Request |
| Profesional inexistente | 404 Not Found |
| Cliente inexistente | 404 Not Found |
| Disponibilidad inexistente | 404 Not Found |
| Disponibilidad ocupada | 409 Conflict |
| Disponibilidad perteneciente a otro profesional | 409 Conflict |
| Superposición o duplicación de horario | 409 Conflict |

Ejemplo de respuesta de conflicto:


```
{
 "mensaje": "La disponibilidad no esta disponible"
}
```


*PUT /turnos/:id*

- **Recurso:** turno.
- **Método HTTP:** PUT.
- **URL:** /turnos/:id.
- **Parámetros de ruta:** id, identificador del turno.
- **Parámetros de consulta:** no tiene.
- **Body:** únicamente el estado atendido.
- **Descripción:** permite solamente la transición reservado → atendido. La disponibilidad permanece ocupada.
- **Ejemplo de solicitud:**


```
PUT http://localhost:3000/turnos/1
Content-Type: application/json

{
 "estado": "atendido"
}
```


- **Código HTTP esperado:** 200 OK.
- **Respuesta correcta:**


```
{
 "id": 1,
```

```
 "profesionalId": 1,
 "clienteId": 1,
 "disponibilidadId": 1,
 "fecha": "2026-10-15",
 "hora": "11:00",
 "estado": "atendido"
}
```


- **Ejemplos de errores:**


```
{
 "mensaje": "El estado debe ser atendido"
}
```


Código: 400 Bad Request para cualquier valor distinto de atendido.


```
{
 "mensaje": "El turno no puede ser atendido"
}
```


Código: 409 Conflict si el turno está cancelado o ya está atendido.

Si el turno no existe, responde 404 Not Found.

*PATCH /turnos/:id/cancelar*

- **Recurso:** turno.
- **Método HTTP:** PATCH.
- **URL:** /turnos/:id/cancelar.
- **Parámetros de ruta:** id, identificador del turno.
- **Parámetros de consulta:** no tiene.
- **Body:** no requiere.
- **Descripción:** cambia el estado a cancelado, libera la disponibilidad relacionada y conserva el turno en el archivo JSON.
- **Ejemplo de solicitud:**

```
PATCH http://localhost:3000/turnos/1/cancelar
```

- **Código HTTP esperado:** 200 OK.
- **Respuesta correcta:**


```
{
 "id": 1,
 "profesionalId": 1,
 "clienteId": 1,
 "disponibilidadId": 1,
 "fecha": "2026-10-15",
 "hora": "11:00",
 "estado": "cancelado"
}
```


- **Ejemplo de error — segunda cancelación:**

```
{
 "mensaje": "El turno ya esta cancelado"
}
```

- **Código HTTP del error:** 409 Conflict.
- **Turno inexistente:** 404 Not Found.

No existe DELETE físico de turnos porque la cancelación conserva el registro.

## 7. Códigos HTTP

| Código | Nombre | Uso en la API |
|---|---|---|
| 200 | OK | Consultas y modificaciones realizadas correctamente. |
| 201 | Created | Creación correcta de profesionales, clientes, disponibilidades o turnos. |
| 204 | No Content | Eliminaciones correctas de recursos que se eliminan físicamente. |
| 400 | Bad Request | Datos inválidos, body incorrecto o filtros inválidos. |
| 404 | Not Found | Registro o relación requerida que no existe. |
| 409 | Conflict | Disponibilidad ocupada, duplicaciones o transiciones no permitidas. |
| 500 | Internal Server Error | Error interno inesperado manejado por Express. |

## 8. Consultas específicas

### Turnos por profesional

```
GET /turnos?profesionalId=1
```

Devuelve los turnos asociados al profesional con ID 1.

### Turnos por cliente

```
GET /turnos?clienteId=1
```

Devuelve los turnos asociados al cliente con ID 1.

### Disponibilidades libres de un profesional y fecha

```
GET /disponibilidades?profesionalId=1&fecha=2026-10-15&disponible=true
```

Devuelve las disponibilidades libres del profesional indicado para esa fecha.

## 9. Validaciones

Las validaciones se realizan dentro de los controladores y no en un middleware personalizado.

Se validan:

- campos obligatorios;
- tipos de datos;
- IDs enteros positivos;

- nombres y apellidos como strings no vacíos;
- formato de fechas y horas;
- valores booleanos de disponibilidad;
- estados permitidos;
- existencia de profesionales, clientes y disponibilidades relacionadas;
- disponibilidad libre antes de reservar;
- pertenencia entre una disponibilidad y su profesional;
- superposición de turnos;
- duplicación de reservas del cliente;
- transiciones permitidas de los estados.

## 10. Manejo de errores

La API responde con códigos diferenciados para:

- datos incorrectos;
- registros inexistentes;
- conflictos de disponibilidad;
- duplicaciones;
- operaciones no permitidas;
- errores internos inesperados.

Los errores inesperados son registrados en consola y procesados por el middleware general de errores de Express, que responde 500 Internal Server Error con:

```
{
  "mensaje": "Error interno del servidor"
}
```

## 11. Persistencia y asincronía

src/data/db.json funciona como persistencia de esta instancia académica.

El archivo src/utils/jsonFile.js centraliza el acceso a los datos. Utiliza fs.promises, funciones async y await para leer y escribir el JSON de manera asíncrona. El código de aplicación no utiliza lectura ni escritura síncrona del archivo.

## 12. POO

Las entidades del dominio están representadas por clases ubicadas en src/models:

- Profesional
- Cliente
- Disponibilidad
- Turno

Estas clases se utilizan al crear nuevos registros. Su responsabilidad es representar los datos de cada entidad; las validaciones y reglas de negocio permanecen en los controladores.

## 13. Pug

La ruta:

```
GET /vista
```

renderiza una plantilla Pug informativa sobre TurnoFlex. La vista muestra el nombre del proyecto, su objetivo, los recursos principales y las tecnologías utilizadas.

No se desarrolló un frontend completo porque no forma parte del alcance del proyecto.

## 14. Pruebas

Los siguientes casos fueron probados con Thunder Client:

| Caso | Método | Endpoint | Resultado esperado |
|---|---|---|---|
| Listar profesionales | GET | /profesionales | 200 |
| Crear profesional | POST | /profesionales | 201 |
| Profesional inexistente | GET | /profesionales/999 | 404 |
| Datos de cliente inválidos | POST | /clientes | 400 |
| Crear disponibilidad | POST | /disponibilidades | 201 |
| Disponibilidad duplicada | POST | /disponibilidades | 409 |
| Crear turno | POST | /turnos | 201 |
| Disponibilidad ocupada | POST | /turnos | 409 |
| Cancelar turno | PATCH | /turnos/1/cancelar | 200 |
| Cancelar nuevamente | PATCH | /turnos/1/cancelar | 409 |
| Atender turno | PUT | /turnos/1 | 200 |
| Atender turno cancelado | PUT | /turnos/1 | 409 |

## 15. Integrantes y responsabilidades

El proyecto fue desarrollado de manera grupal, distribuyendo las tareas entre los integrantes y realizando revisiones conjuntas antes de la entrega.

**Mateo Israel Adan** — Integración general del proyecto; desarrollo y revisión de la lógica de turnos; validación de reglas de negocio; pruebas de integración de la API.

**Baca Fernando Ezequiel** — Desarrollo y revisión de los módulos de Profesionales y Clientes; validaciones de datos y operaciones CRUD; pruebas de respuestas y códigos HTTP.

**Lamas Carmen Leonela** — Desarrollo y revisión del módulo de Disponibilidades; implementación y verificación de filtros mediante parámetros de consulta; pruebas de horarios disponibles y registros relacionados.

**Mata Botana Raúl Alberto** — Desarrollo y revisión del módulo de Turnos; verificación de reservas, estados, cancelaciones y manejo de conflictos; pruebas de reglas de superposición y disponibilidad.

**Yrala Marcelo** — Revisión de la estructura general del proyecto; verificación de middleware, manejo de errores y vista Pug; revisión de documentación, README y pruebas finales.

### Trabajo conjunto

Además de la distribución principal, todos los integrantes participaron en la revisión general del sistema, análisis de las consignas, pruebas funcionales y preparación de la entrega final.

## 16. Uso de herramientas de IA

Se utilizaron herramientas de inteligencia artificial como apoyo para la revisión del código, detección de errores, organización de la documentación y asistencia durante las pruebas. Las decisiones de implementación, revisión y comprensión del funcionamiento del proyecto quedaron a cargo del equipo.

## 17. Fuentes consultadas


- Material de la materia Desarrollo Backend.
- Documentación oficial de Node.js.
- Documentación oficial de Express.
- Documentación oficial de Pug.